L12 - 预训练 Pretrain

> *"让模型学会词语接龙"*

📌 本节目标

1. 理解预训练的核心目标：Next Token Prediction
2. 掌握交叉熵损失函数的数学原理
3. 理解学习率调度（余弦退火 + warmup）的原理和作用
4. 掌握混合精度训练（AMP）和梯度裁剪的概念
5. 能读懂 MiniMind 的 `train_pretrain.py` 源码

📚 前置知识

- L11 数据处理流水线
- 基本的微积分知识（求导、链式法则）
- 了解梯度下降的基本原理

---

1. 预训练的目标：Next Token Prediction

1.1 什么是预训练？

预训练是 LLM 训练的第一阶段，目标是让模型**从海量文本中学习语言规律和世界知识**。

打个比方：预训练就像让一个孩子读遍所有的书——百科全书、小说、论文、新闻……读完之后，这个孩子虽然还不会"对话"，但已经掌握了丰富的语言能力和知识。

1.2 Next Token Prediction

预训练的任务出奇地简单——**给定前面的所有 token，预测下一个 token**。

```
输入："人工智能是计算机科学的一个"
目标："分"
```

```
输入："人工智能是计算机科学的一个分"
目标："支"
```

就像一个超级词语接龙游戏。模型要学会在所有可能的上下文中，预测最合理的下一个 token。

1.3 自回归语言模型

这种"基于前文预测下一个 token"的方式叫做**自回归**（Autoregressive）。形式化地表达：

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

模型把整个文本的联合概率分解为一系列条件概率的乘积。训练时，每个位置都在做一次分类任务：在词表大小的候选中，选出正确的下一个 token。

---

2. 交叉熵损失函数

2.1 直觉理解

模型每预测一个 token，会输出一个概率分布（对词表中每个 token 的概率预测）。我们希望**正确 token 的概率尽可能高**。

交叉熵损失衡量的就是模型预测的分布和真实分布之间的"差距"。

2.2 数学公式

对于单个 token 位置，交叉熵损失为：

$$\mathcal{L} = -\log P(x_{\text{correct}})$$

其中 $P(x_{\text{correct}})$ 是模型对正确 token 的预测概率。

直觉： - 如果模型预测正确 token 的概率为 0.9，loss = $-\log(0.9) = 0.105$，很小 - 如果模型预测正确 token 的概率为 0.01，loss = $-\log(0.01) = 4.605$，很大

对整个序列取平均：

$$\mathcal{L}_{\text{total}} = -\frac{1}{N}\sum_{t=1}^{N} \log P(x_t \mid x_1, \ldots, x_{t-1})$$

2.3 与 Perplexity 的关系

困惑度（Perplexity, PPL）是评估语言模型的经典指标：

PPL = $e^{\mathcal{L}}$

PPL 可以直观理解为：模型在每个位置"平均困惑于"多少个候选 token。PPL 越低，模型越好。

2.4 PyTorch 中的实现

```
import torch.nn.functional as F

# logits: [batch_size, seq_len, vocab_size] - 模型输出
# labels: [batch_size, seq_len] - 真实 token ids
loss = F.cross_entropy(
    logits.view(-1, vocab_size),  # 展平为 [batch*seq, vocab]
    labels.view(-1),              # 展平为 [batch*seq]
    ignore_index=-100             # 忽略 padding 位置（标记为-100）
)
```

`ignore_index=-100` 是 PyTorch 的约定：labels 中值为 -100 的位置不参与 loss 计算，这就是 Loss Mask 在 PyTorch 中的实现方式。

---

3. 预训练的完整流程

3.1 训练循环

```
for each epoch:
    for each batch in dataloader:
        1. 将 input_ids 送入模型 → 得到 logits
        2. 计算 cross_entropy loss
        3. loss.backward() → 计算梯度
        4. 梯度裁剪
        5. optimizer.step() → 更新参数
        6. scheduler.step() → 更新学习率
        7. optimizer.zero_grad() → 清零梯度
```

3.2 输入和标签的构造

预训练时，输入和标签有一个 token 的偏移：

```
原始序列:  [A, B, C, D, E]
输入 (x):  [A, B, C, D]    → 模型的输入
标签 (y):  [B, C, D, E]    → 模型需要预测的目标
```

代码中通常这样实现：

```
input_ids = token_ids[:-1]   # 去掉最后一个
labels = token_ids[1:]       # 去掉第一个
```

这样每个位置 t 的训练信号就是：给定 $x_1 \ldots x_t$，预测 x_{t+1}。

---

4. 学习率调度：余弦退火 + Warmup

4.1 为什么需要学习率调度?

固定学习率训练存在问题： - 太大：训练不稳定，loss 震荡 - 太小：训练太慢 - 训练早期和后期对学习率的需求不同

4.2 Warmup 阶段

训练刚开始时，模型参数是随机初始化的，梯度方向不可靠。如果一上来就用大学习率，参数可能"飞了"。

Warmup：在前几百步中，让学习率从 0 线性增长到目标值。

```
if step < warmup_steps:
    lr = max_lr * step / warmup_steps
```

4.3 余弦退火（Cosine Annealing）

Warmup 之后，学习率按余弦函数逐渐衰减：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\frac{t - t_w}{T - t_w} \cdot \pi))$$

其中： - $\eta_{\max}$：最大学习率 - $\eta_{\min}$：最小学习率 - t_w：warmup 步数 - T：总训练步数

为什么用余弦而不是线性衰减? 余弦退火在训练中期衰减较慢，给模型更多时间学习；在训练末期加速衰减，帮助模型精细调整。

4.4 学习率曲线示意

```
学习率
  |    /\
  |   /  \
  |  /    \
  | /      \____
```

```
  |/            \
  +-------------→ 训练步数
  ↑warmup  ↑cosine decay
```

4.5 MiniMind 的实现

MiniMind 在 `train_pretrain.py` 中使用自定义的学习率调度器：

```
def get_lr(current_step, warmup_steps, max_steps, max_lr, min_lr):
    if current_step < warmup_steps:
        return max_lr * current_step / warmup_steps
    if current_step > max_steps:
        return min_lr
    decay_ratio = (current_step - warmup_steps) / (max_steps - warmup_steps)
    coeff = 0.5 * (1.0 + math.cos(math.pi * decay_ratio))
    return min_lr + coeff * (max_lr - min_lr)
```

---

5. 混合精度训练（AMP）

5.1 什么是混合精度?

默认情况下，模型参数和计算使用 FP32（32位浮点数）。混合精度训练在**计算时使用 FP16/BF16，存储主权重时保留 FP32**，从而：

1. **减少显存占用**：FP16 占用空间是 FP32 的一半
2. **加速计算**：现代 GPU（如 3090）有专门的 FP16 计算单元（Tensor Core）
3. **保持精度**：关键操作仍用 FP32

5.2 FP16 vs BF16

特性	FP16	BF16
指数位	5 位	8 位
尾数位	10 位	7 位
数值范围	较小	与 FP32 相同
精度	较高	较低
适用场景	需要 loss scaling	更稳定，推荐使用

BF16 的数值范围与 FP32 相同，不容易出现溢出（overflow），是目前训练 LLM 的首选。

5.3 PyTorch AMP 的使用

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for batch in dataloader:
    with autocast(dtype=torch.bfloat16):
        logits = model(input_ids)
        loss = compute_loss(logits, labels)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad()
```

`GradScaler` 在使用 FP16 时非常重要，它通过放大 loss 来防止梯度下溢（underflow）。使用 BF16 时通常可以省略 scaler。

---

6. 梯度裁剪（Gradient Clipping）

6.1 为什么需要梯度裁剪?

训练过程中，某些 batch 可能产生异常大的梯度（梯度爆炸），导致参数更新过大，训练崩溃。

6.2 实现方式

最常用的是**按范数裁剪**：如果所有参数梯度的 L2 范数超过阈值，则等比例缩小：

$$\mathbf{g} \leftarrow \frac{\text{max_norm}}{||\mathbf{g}||_2} \cdot \mathbf{g}, \quad \text{if } ||\mathbf{g}||_2 > \text{max_norm}$$

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

MiniMind 使用 `max_norm=1.0`，这是业界常用的默认值。

6.3 梯度裁剪 vs 梯度归一化

- **梯度裁剪**：只在梯度过大时裁剪，正常梯度不受影响
- **梯度归一化**：总是将梯度缩放到固定范数

梯度裁剪更常用，因为它不会影响正常的梯度更新。

---

7. 训练开销与检查点

7.1 MiniMind 训练开销

MiniMind 64M 参数模型在 NVIDIA 3090（24GB）上的训练时间参考：

训练阶段	数据集	大约时间
预训练（小）	pretrain_t2t_mini.jsonl	~1.21h
预训练（全量）	pretrain_t2t.jsonl	~10h

7.2 检查点保存

训练过程中定期保存模型权重（checkpoint），用于： 1. **断点续训**：训练中断后从最近的 checkpoint 恢复 2. **选择最优模型**：训练结束后选 loss 最低的 checkpoint 3. **分析训练过程**：对比不同阶段的模型能力

```
if step % save_interval == 0:
    torch.save({
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'step': step,
        'loss': loss.item(),
    }, f'checkpoint_step_{step}.pt')
```

7.3 断点续训

```
checkpoint = torch.load('checkpoint_step_1000.pt')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
start_step = checkpoint['step']
```

---

8. 预训练后的模型能做什么？

预训练完成后，模型学会了： - 语法和语言结构 - 常识和世界知识 - 基本的逻辑推理能力

但是，预训练模型**不会对话**！如果你给它输入"什么是机器学习？"，它可能会续写成一篇论文风格的文章，而不是给你一个结构化的回答。

```
输入："什么是机器学习？"
预训练模型输出："机器学习是近年来备受关注的研究领域之一。在过去的十年中..."
                （续写模式，而非对话模式）

SFT 后的模型输出："机器学习是人工智能的一个子领域，它让计算机能够从数据中
                    自动学习规律，而不需要被显式编程。主要分为三类：..."
                （对话模式，结构化回答）
```

这就是为什么预训练之后还需要 SFT（监督微调）。

---

9. 训练 Loss 曲线解读

一条健康的预训练 loss 曲线应该是：

```
Loss
  |
8 |*
  | *
6 |  *
  |   **
4 |     ***
  |        ****
2 |            **********
  |                     **********
  +------------------------------------> Steps
```

- **快速下降阶段**：模型快速学习基本的语言模式
- **缓慢下降阶段**：学习更细粒度的语言规律
- **趋于平稳**：模型接近收敛

异常情况： - **loss 突然上升**：学习率太大，或数据有问题 - **loss 持续不下降**：学习率太小，或模型太小 - **loss 震荡剧烈**：batch size 太小，梯度估计噪声大

---

10. MiniMind 源码解读

10.1 关键文件

- `trainer/train_pretrain.py`：预训练主脚本

10.2 核心训练逻辑

`train_pretrain.py` 的主要流程：

```
# 1. 加载配置和模型
model = MiniMindForCausalLM(config)

# 2. 构建数据集和 DataLoader
dataset = PretrainDataset(data_path, tokenizer, max_seq_len)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# 3. 设置优化器
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

# 4. 训练循环
for epoch in range(num_epochs):
    for step, batch in enumerate(dataloader):
        # 更新学习率
        lr = get_lr(step, warmup_steps, max_steps, max_lr, min_lr)
        for param_group in optimizer.param_groups:
            param_group['lr'] = lr

        # 前向传播（混合精度）
        with autocast():
            logits = model(input_ids)
            loss = cross_entropy(logits, labels)

        # 反向传播
        loss.backward()

        # 梯度裁剪
        clip_grad_norm_(model.parameters(), max_norm=1.0)

        # 参数更新
        optimizer.step()
        optimizer.zero_grad()

        # 保存检查点
        if step % save_interval == 0:
            save_checkpoint(model, optimizer, step)
```

10.3 多卡训练支持

MiniMind 支持 DDP（Distributed Data Parallel）和 DeepSpeed：

```
# 单卡
python trainer/train_pretrain.py

# DDP 多卡
torchrun --nproc_per_node=2 trainer/train_pretrain.py

# DeepSpeed
deepspeed trainer/train_pretrain.py --deepspeed ds_config.json
```

🎤 面试考点

Q1: 预训练的目标函数是什么？

答：预训练使用 Next Token Prediction 任务，目标函数是交叉熵损失：$\mathcal{L} = -\frac{1}{N}\sum_{t=1}^{N} \log P(x_t \mid x_1, ..., x_{t-1})$。模型在每个位置预测下一个 token 的概率分布，通过最小化交叉熵损失来训练。

Q2: 学习率 Warmup 的作用是什么？

答：训练初期模型参数随机，梯度方向不可靠。直接使用大学习率可能导致参数更新过大，模型"飞了"。Warmup 让学习率从 0 线性增长到目标值，给模型一个"热身"期，使参数先小步调整到合理区域，再加速学习。

Q3: 混合精度训练的原理是什么？有什么好处？

答：混合精度训练在前向和反向传播时使用低精度（FP16/BF16），在参数存储和更新时使用 FP32。好处：(1) 显存减半；(2) 利用 Tensor Core 加速计算 2-3 倍；(3) 通过 master weight 保持训练精度。使用 FP16 时需要 GradScaler 防止梯度下溢。

Q4: 为什么需要梯度裁剪？

答：深度网络训练中可能出现梯度爆炸（gradient explosion），导致参数更新过大、训练崩溃。梯度裁剪设置一个最大范数阈值，当梯度的 L2 范数超过阈值时，等比例缩小梯度，保持更新方向不变但限制步长。常用阈值为 1.0。

Q5: 余弦退火相比线性衰减有什么优势？

答：余弦退火在训练中期学习率衰减较慢，给模型更多时间学习复杂模式；在训练末期加速衰减，帮助模型精细收敛。相比线性衰减，余弦退火的 loss 曲线通常更平滑，最终收敛效果更好。

Q6: 预训练后的模型有什么能力？有什么不足？

答：预训练后的模型掌握了语法、常识、世界知识和基本推理能力，能进行文本续写。但它不具备对话能力——无法理解指令、不会按格式回答问题。需要通过 SFT 让模型学会"对话"，通过 RLHF/DPO 进行偏好对齐。

✅ 自测题

1. 写出交叉熵损失函数的完整公式，并解释每个符号的含义。
2. 解释 warmup + cosine annealing 学习率调度策略，画出其曲线。
3. FP16 和 BF16 有什么区别？为什么现在更推荐 BF16？
4. 梯度裁剪和梯度归一化有什么区别？
5. 预训练模型和 SFT 模型在输入"你好"时，分别会有什么样的输出？为什么？

⏭️ 下一节预告

L13 - 监督微调 SFT：预训练让模型学会了"语言"，但还不会"说话"。下一节我们将学习如何通过监督微调，让模型从"百科全书"变成"对话助手"，深入理解 chat_template、Loss Mask 的实现，以及 MiniMind 的 SFT 训练流程。